МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ Федеральное государственное бюджетное образовательное учреждение высшего образования

«СИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ТЕЛЕКОММУНИКАЦИЙ И ИНФОРМАТИКИ»
(СибГУТИ)
Институт вычислительной техники
Кафедра вычислительных систем

ОТЧЕТ

по практической работе № 3
по дисциплине «Основы разработки на Java»

Выполнил: студент гр. ИС-541
Устюжанин Д. К.

Проверил: доцент кафедры ВС
Нигматулин Р. Р.

Новосибирск 2026

Содержание

Содержание **2**
1. Введение **3**
1.1 Цель работы 3
1.3 Общая предметная модель 3
2. Уровень I. Полиморфизм **4**
2.1 Задание 1. Один контракт, две реализации 4
2.2 Задание 2. Анализ Dynamic Dispatch и Overload Resolution 4
2.3 Задание 3. Полиморфный массив 5
3. Уровень II. Абстракция и наследование **6**
3.1 Задание 4. Абстрактный базовый класс и шаблонный метод 6
3.2 Задание 5. Выбор формы типа 6
3.3 Задание 6. Переопределение методов и аннотация @Override 7
3.4 Задание 7. Композиция вместо наследования (Декоратор) 7
4. Уровень III. Контракты подтипов **9**
4.2 Задание 9. Политика допуска как стратегия 9
4.3 Задание 10. Разрешение конфликта default-методов 10
5. Уровень IV. Закрытые и открытые иерархии **11**
5.1 Задание 11. Равенство в иерархии наследования 11
5.2 Задание 12. Sealed-иерархия результатов 12
6. Уровень V. Проектирование **13**
6.1 Задание 13. Замена Type Code полиморфизмом 13
6.2 Задание 14. Декомпозиция независимых осей вариативности 14
6.3 Задание 15. Расширяемый движок правил CourseHub 14
7. Заключение **16**
Приложение A. Полные листинги программы **17**

1. Введение

В рамках данной практической работы рассматриваются ключевые концепции объектно-ориентированного программирования и проектирования на языке Java. В отчете представлены решения задач, охватывающих использование механизмов полиморфизма, интерфейсов, абстрактных классов и наследования, а также фундаментальных паттернов проектирования, таких как «Стратегия», «Шаблонный метод» и «Декоратор». Практическая реализация и анализ решений выполняются на примере сквозной предметной модели учебного центра CourseHub.

Исходный код и полные листинги программы доступны в репозитории GitHub: https://github.com/Kuso0taku/sibsutis-java.

1.1 Цель работы

Освоить объектно-ориентированные механизмы языка Java, лежащие в основе построения гибких и расширяемых программных систем: интерфейсы и динамическую диспетчеризацию, абстрактные классы и шаблонный метод, правила переопределения и перегрузки, принцип подстановки Барбары Лисков (LSP), паттерны проектирования «Стратегия» и «Декоратор», закрытые и открытые иерархии (sealed), а также методы устранения типовых ошибок проектирования.

1.3 Общая предметная модель

Все задания выполнены на основе сквозной предметной модели учебного центра **CourseHub**. Общие классы вынесены в модуль `src` и переиспользуются во всех заданиях без дублирования кода.

Основными сущностями предметной области являются:

- **`src.Course`** — предметная модель учебного курса (содержит идентификатор, наименование, длительность, статус, вместимость и список записанных студентов);
- **`src.Student`** — предметная модель студента (содержит идентификатор, имя, возраст и признак прохождения пререквизитов).

2. Уровень I. Полиморфизм

2.1 Задание 1. Один контракт, две реализации

В соответствии с условием задания объявлен интерфейс `CourseFormatter` с методом `format(Course)`. Созданы две реализации: `CompactCourseFormatter` и `DetailedCourseFormatter`.

При вызове методов через переменную интерфейсного типа компилятору на этапе сборки (compile-time) известен только контракт интерфейса CourseFormatter. Конкретный метод вызова определяется во время выполнения (runtime) по фактическому классу объекта на основе механизма динамической диспетчеризации.

Листинг 1 — Объявление интерфейса CourseFormatter

```
package task1;
import src.Course;
public interface CourseFormatter {
    String format(Course course);
}
```

Листинг 2 — Демонстрация вызова реализаций

```
package task1;
import src.Course;
public class Task1Demo {
    public static void main(String[] args) {
        Course course = new Course(1, "Java", 10);
        CourseFormatter compact = new CompactCourseFormatter();
        CourseFormatter detailed = new DetailedCourseFormatter();
        System.out.println(compact.format(course));
        System.out.println(detailed.format(course));
    }
}
```

2.2 Задание 2. Анализ Dynamic Dispatch и Overload Resolution

Разграничены понятия разрешения перегрузки (overload resolution) и динамической диспетчеризации (override dispatch):

- **Overload resolution** выполняется компилятором на этапе компиляции по статическим (compile-time) типам аргументов;
- **Override dispatch** выполняется виртуальной машиной Java (JVM) во время выполнения по фактическому классу объекта.

Таблица 2 — Результаты выполнения вызовов

Вызов	Compile-time тип	Runtime класс	Выполняемый метод
`asInterface.send(...)`	`NotificationSender`	`EmailSender`	`EmailSender.send` (Override dispatch)
`console.send(...)`	`NotificationSender`	`ConsoleSender`	`ConsoleSender.send` (Override dispatch)
`describe(asInterface)`	`NotificationSender`	`EmailSender`	`describe(NotificationSender)` (Overload)
`describe(asConcrete)`	`EmailSender`	`EmailSender`	`describe(EmailSender)` (Overload)

2.3 Задание 3. Полиморфный массив

Создан массив базового интерфейсного типа `CourseFormatter[]`, содержащий объекты трех реализаций (`CompactCourseFormatter`, `DetailedCourseFormatter`, `HtmlCourseFormatter`). Обработка элементов осуществляется в едином цикле без операторов условного ветвления if или проверки типов `instanceof`. Добавление новой реализации не требует изменения клиентского кода.

Листинг 3 — Демонстрация работы с полиморфным массивом

```
package task3;
import src.Course;
import task1.CompactCourseFormatter;
import task1.CourseFormatter;
import task1.DetailedCourseFormatter;
public class Task3Demo {
    public static void main(String[] args) {
        Course course = new Course(1, "Java", 10);
        CourseFormatter[] formatters = new CourseFormatter[] {
            new CompactCourseFormatter(),
            new DetailedCourseFormatter(),
            new HtmlCourseFormatter()
        };
        for (CourseFormatter formatter : formatters) {
            System.out.println(formatter.format(course));
        }
    }
}
```

3. Уровень II. Абстракция и наследование

3.1 Задание 4. Абстрактный базовый класс и шаблонный метод

Реализован абстрактный класс `AbstractNotificationSender` с применением паттерна **«Шаблонный метод» (Template Method)**:

- Публичный метод `send()` объявлен как `final`, фиксируя общий инвариант алгоритма (валидацию адресата);
- Защищенный абстрактный метод doSend() представляет собой единственную точку расширения для подклассов.

Листинг 3 — Базовый абстрактный класс отправителя уведомлений

```
package task4;
import task2.NotificationSender;
public abstract class AbstractNotificationSender implements
NotificationSender {
    @Override
    public final void send(String recipient, String message) {
        if (recipient == null || recipient.isBlank()) {
            throw new IllegalArgumentException("recipient must not be
blank");
        }
        doSend(recipient, message);
    }
    protected abstract void doSend(String recipient, String message);
}
```

3.2 Задание 5. Выбор формы типа

Таблица 3 — Мотивировка выбора конструкций языка Java

Понятие	Выбранная конструкция	Мотивировка выбора
`Identifiable`	`interface`	Описывает поведение/контракт наличия `id()`, без состояния.
`CoursePolicy`	`interface`	Чистая стратегия без собственного состояния.
`BaseEntity`	`abstract class`	Содержит общее состояние (`id`) и

Понятие	Выбранная конструкция	Мотивировка выбора
		реализацию `equals`/`hashCode`.
`Formatter`	`interface`	Контракт преобразования данных без состояния.
`ClockProvider`	`interface`	Функциональный интерфейс для поставщика времени.

3.3 Задание 6. Переопределение методов и аннотация `@Override`

Аннотация @Override гарантирует, что помеченный метод действительно переопределяет метод базового типа. При несовпадении сигнатур без аннотации метод не переопределяется, а становится перегруженным.

Листинг 4 — Корректное переопределение методов

```
package task6;
class FixedBase {
    void send(String to) { System.out.println("base.send"); }
    protected void log(String msg) { System.out.println("base.log"); }
    Number value() { return 1; }
}
class Fixed extends FixedBase {
    @Override
    void send(String to) { System.out.println("fixed.send to " + to); }
    @Override
    protected void log(String msg) { System.out.println("fixed.log: " +
msg); }
    @Override
    Integer value() { return 2; }
}
```

3.4 Задание 7. Композиция вместо наследования (Декоратор)

Для предотвращения комбинаторного взрыва классов при добавлении функций логирования и сбора метрик использован паттерн **«Декоратор»**. Внешний декоратор первым перехватывает вызов метода, благодаря чему порядок обертывания напрямую влияет на последовательность выполнения гибкого поведения.

Листинг 5 — Декораторы сбора метрик и логирования

```
package task7;
import task2.NotificationSender;
public class LoggingDecorator implements NotificationSender {
    private final NotificationSender delegate;
    public LoggingDecorator(NotificationSender delegate) { this.delegate
= delegate; }
    @Override
    public void send(String recipient, String message) {
        System.out.println("[LOG] Sending message to " + recipient);
        delegate.send(recipient, message);
    }
}
public class MetricsDecorator implements NotificationSender {
    private final NotificationSender delegate;
    public MetricsDecorator(NotificationSender delegate) { this.delegate
= delegate; }
    @Override
    public void send(String recipient, String message) {
        System.out.println("[METRICS] Incrementing notification count");
        delegate.send(recipient, message);
    }
}
```

Листинг 6 — Демонстрация композиции декораторов

```
package task7;
import task2.EmailSender;
import task2.NotificationSender;
public class Task7Demo {
    public static void main(String[] args) {
        NotificationSender sender1 = new LoggingDecorator(new
MetricsDecorator(new EmailSender()));
        NotificationSender sender2 = new MetricsDecorator(new
LoggingDecorator(new EmailSender()));
        sender1.send("user@test.com", "Hello");
        sender2.send("user@test.com", "World");
    }
}
```

4. Уровень III. Контракты подтипов

4.2 Задание 9. Политика допуска как стратегия

Реализован паттерн **«Стратегия»** для проверки условий записи на курс (`EnrollmentPolicy`). Внедрение зависимости производится через конструктор сервиса EnrollmentService. Добавление новых правил не требует модификации существующего кода сервиса.

Листинг 7 — Реализации политик допуска и сервиса зачисления

```
package task9;
import src.Course;
import src.Student;
public interface EnrollmentPolicy {
    boolean canEnroll(Student student, Course course);
}
public class AgePolicy implements EnrollmentPolicy {
    private final int minAge;
    public AgePolicy(int minAge) { this.minAge = minAge; }
    @Override
    public boolean canEnroll(Student student, Course course) { return
student.age() >= minAge; }
}
public class PrerequisitePolicy implements EnrollmentPolicy {
    @Override
    public boolean canEnroll(Student student, Course course) { return
student.hasPrerequisites(); }
}
public class CapacityPolicy implements EnrollmentPolicy {
    @Override
    public boolean canEnroll(Student student, Course course) { return
course.students().size() < course.capacity(); }
}
public class EnrollmentService {
    private final EnrollmentPolicy policy;
    public EnrollmentService(EnrollmentPolicy policy) { this.policy =
policy; }
    public boolean enroll(Student student, Course course) { return
policy.canEnroll(student, course); }
}
```

Листинг 8 — Демонстрация работы сервиса зачисления

```
package task9;
import src.Course;
import src.Student;
```

```
public class Task9Demo {
    public static void main(String[] args) {
        Student student = new Student(1, "Alice", 20, true);
        Course course = new Course(101, "Java Advanced", 10);
        EnrollmentService service = new EnrollmentService(new
AgePolicy(18));
        System.out.println("Enrolled: " + service.enroll(student,
course));
    }
}
```

4.3 Задание 10. Разрешение конфликта default-методов

При наследовании классом двух интерфейсов с одинаковыми default-методами возникает ошибка компиляции из-за неоднозначности. Конфликт разрешается явным переопределением метода в целевом классе.

Листинг 5 — Разрешение конфликта default-методов

```
package task10;
interface LoggerA { default void log(String msg) { System.out.println("A:
" + msg); } }
interface LoggerB { default void log(String msg) { System.out.println("B:
" + msg); } }
public class CustomLogger implements LoggerA, LoggerB {
    @Override
    public void log(String msg) { LoggerA.super.log(msg); }
}
```

5. Уровень IV. Закрытые и открытые иерархии

5.1 Задание 11. Равенство в иерархии наследования

Добавление новых полей в подклассах (например, цвета в ColoredPoint extends Point) нарушает свойство симметричности контракта `equals()`. Для обеспечения корректности предложены два решения:

1. Использование строгого сравнения классов (`getClass()`);
2. Использование композиции вместо наследования (создание класса-обертки или `record`).

Листинг 9 — Подходы к реализации равенства в иерархии

```
package task11;
import java.util.Objects;
public class Point {
    private final int x, y;
    public Point(int x, int y) { this.x = x; this.y = y; }
    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }
}
public class ColoredPoint extends Point {
    private final String color;
    public ColoredPoint(int x, int y, String color) { super(x, y);
this.color = color; }
}
public class StrictPoint {
    private final int x, y;
    public StrictPoint(int x, int y) { this.x = x; this.y = y; }
    @Override
    public boolean equals(Object o) {
        if (o == null || getClass() != o.getClass()) return false;
        StrictPoint p = (StrictPoint) o;
        return x == p.x && y == p.y;
    }
}
public class StrictColoredPoint extends StrictPoint {
    private final String color;
    public StrictColoredPoint(int x, int y, String color) { super(x, y);
this.color = color; }
}
public record PlainPoint(int x, int y) {}
public record Colored<T>(T point, String color) {}
```

Листинг 10 — Демонстрация равенства и нарушения симметричности

```
package task11;
public class Task11Demo {
    public static void main(String[] args) {
        Point p = new Point(1, 2);
        ColoredPoint cp = new ColoredPoint(1, 2, "RED");
        System.out.println("p.equals(cp): " + p.equals(cp));
        System.out.println("cp.equals(p): " + cp.equals(p));
    }
}
```

5.2 Задание 12. Sealed-иерархия результатов

Создана закрытая иерархия типов EnrollmentResult с использованием ключевого слова sealed.

Листинг 6 — Sealed-интерфейс результата зачисления

```
package task12;
public sealed interface EnrollmentResult
    permits Accepted, Rejected, WaitListed, Deferred {
}
```

Использование `sealed`-иерархии позволяет применять исчерпывающий оператор `switch` без использования секции `default`. Добавление нового разрешенного типа `Deferred` контролируется компилятором, который требует обработать новый тип во всех выражениях `switch`.

6. Уровень V. Проектирование

6.1 Задание 13. Замена Type Code полиморфизмом

Управленческий кодовый признак (`type code`) и логика с оператором `switch` заменены на иерархию реализаций интерфейса NotificationChannel. Выбор поведения перенесен на уровень динамической диспетчеризации.

Листинг 11 — Исходный код с условным оператором по типу

```
package task13;
public class TypeCodeBefore {
    public void send(String type, String message) {
        switch (type) {
            case "EMAIL" -> System.out.println("Email: " + message);
            case "SMS" -> System.out.println("SMS: " + message);
            case "PUSH" -> System.out.println("Push: " + message);
            default -> throw new IllegalArgumentException("Unknown type:
" + type);
        }
    }
}
```

Листинг 12 — Замена Type Code полиморфными каналами

```
package task13;
public interface NotificationChannel {
    void send(String message);
}
public class EmailChannel implements NotificationChannel {
    @Override
    public void send(String message) { System.out.println("Email: " +
message); }
}
public class SmsChannel implements NotificationChannel {
    @Override
    public void send(String message) { System.out.println("SMS: " +
message); }
}
public class PushChannel implements NotificationChannel {
    @Override
    public void send(String message) { System.out.println("Push: " +
message); }
}
public class Notifier {
    private final NotificationChannel channel;
```

```
    public Notifier(NotificationChannel channel) { this.channel =
channel; }
    public void notify(String message) { channel.send(message); }
}
```

Листинг 13 — Демонстрация отправки сообщений через каналы

```
package task13;
public class Task13Demo {
    public static void main(String[] args) {
        Notifier notifier = new Notifier(new EmailChannel());
        notifier.notify("Polymorphic notification");
    }
}
```

6.2 Задание 14. Декомпозиция независимых осей вариативности

Для предотвращения комбинаторного взрыва классов независимые оси разделены на отдельные компоненты CourseFormat и PaymentPolicy. Компоновка выполняется через композицию в классе CourseOffer.

Листинг 7 — Декомпозиция через композицию

```
package task14;
public class CourseOffer {
    private final CourseFormat format;
    private final PaymentPolicy payment;
    public CourseOffer(CourseFormat format, PaymentPolicy payment) {
        this.format = format;
        this.payment = payment;
    }
    public String describe(int basePrice) {
        return format.describe() + " / " + payment.price(basePrice) + "
RUB";
    }
}
```

6.3 Задание 15. Расширяемый движок правил CourseHub

Спроектирован движок проверки правил зачисления `EnrollmentEngine`.

- **Правила (`Rule`)** представляют собой открытые стратегии;
- **Результат (`EnrollmentResult`)** является закрытой `sealed`-иерархией;
- Композиции AllOf и AnyOf позволяют строить вложенные древовидные структуры правил без изменения кода сервиса.

Листинг 8 — Интерфейс правила проверки

```
package task15;
import src.Course;
import src.Student;
import task12.EnrollmentResult;
public interface Rule {
    EnrollmentResult check(Student student, Course course);
}
```

7. Заключение

В ходе выполнения практической работы были успешно освоены ключевые объектно-ориентированные механизмы языка Java:

1. Изучено разделение понятий статического связывания (перегрузка) и динамической диспетчеризации (переопределение).
2. Продемонстрированы преимущества композиции перед наследованием для предотвращения комбинаторного взрыва классов и гибкого управления поведением.
3. Освоены механизмы построения строгих контрактов подтипов с соблюдением принципов SOLID (LSP, ISP, DIP).
4. Изучено применение закрытых иерархий (`sealed`), обеспечивающих контроль над полнотой обработки данных на этапе компиляции.

Приложение А. Полные листинги программы

Листинг 12 — src/Course.java (задания 1, 3, 8, 9, 13, 15)

```
package src;
public class Course {
    public enum Status { OPEN, CLOSED }
    private final long id;
    private final String title;
    private int durationHours;
    private Status status = Status.OPEN;
    private int capacity = 30;
    private boolean hasPrerequisite;
    private final java.util.LinkedHashSet<Long> enrolled = new
java.util.LinkedHashSet<>();
    public Course(long id, String title, int durationHours) {
        if (id <= 0) throw new IllegalArgumentException("id must be >
0");
        if (title == null || title.isBlank()) throw new
IllegalArgumentException("title must not be blank");
        if (durationHours <= 0) throw new
IllegalArgumentException("durationHours must be > 0");
        this.id = id;
        this.title = title.trim();
        this.durationHours = durationHours;
    }
    public long id() { return id; }
    public String title() { return title; }
    public int durationHours() { return durationHours; }
    public void setStatus(Status status) { this.status = status; }
    public Status status() { return status; }
    public void setCapacity(int capacity) { this.capacity = capacity; }
    public int capacity() { return capacity; }
    public void setHasPrerequisite(boolean has) { this.hasPrerequisite =
has; }
    public boolean hasPrerequisite() { return hasPrerequisite; }
    public java.util.Set<Long> enrolledStudents() {
        return java.util.Collections.unmodifiableSet(enrolled);
    }
    public void addEnrolled(long studentId) { enrolled.add(studentId); }
    public int enrolledCount() { return enrolled.size(); }
    public int seatsLeft() { return capacity - enrolledCount(); }
}
```

Листинг 13 — src/Student.java (задания 9, 15)

```
package src;
public class Student {
    private final long id;
```

```
    private final String name;
    private final int age;
    private final boolean hasPrerequisite;
    public Student(long id, String name, int age, boolean
hasPrerequisite) {
        if (id <= 0) throw new IllegalArgumentException("id must be >
0");
        if (name == null || name.isBlank()) throw new
IllegalArgumentException("name must not be blank");
        if (age <= 0) throw new IllegalArgumentException("age must be >
0");
        this.id = id;
        this.name = name.trim();
        this.age = age;
        this.hasPrerequisite = hasPrerequisite;
    }
    public long id() { return id; }
    public String name() { return name; }
    public int age() { return age; }
    public boolean hasPrerequisite() { return hasPrerequisite; }
}
```

Листинг 14 — task1/CompactCourseFormatter.java (задания 1, 3)

```
package task1;
import src.Course;
public class CompactCourseFormatter implements CourseFormatter {
    @Override
    public String format(Course course) {
        return course.id() + " " + course.title();
    }
}
```

Листинг 15 — task1/DetailedCourseFormatter.java (задания 1, 3)

```
package task1;
import src.Course;
public class DetailedCourseFormatter implements CourseFormatter {
    @Override
    public String format(Course course) {
        return course.title() + " (" + course.durationHours() + " h, " +
course.status() + ")";
    }
}
```

Листинг 16 — task2/NotificationSender.java (задания 2, 4, 6, 7)

```
package task2;
```

```
public interface NotificationSender {
    void send(String recipient, String message);
}
```

Листинг 17 — task2/EmailSender.java (задания 2, 6, 7)

```
package task2;
public class EmailSender implements NotificationSender {
    @Override
    public void send(String recipient, String message) {
        System.out.println("email to " + recipient + ": " + message);
    }
}
```

Листинг 18 — task2/ConsoleSender.java (задание 2)

```
package task2;
public class ConsoleSender implements NotificationSender {
    @Override
    public void send(String recipient, String message) {
        System.out.println("console[" + recipient + "]: " + message);
    }
}
```

Листинг 19 — task3/HtmlCourseFormatter.java (задание 3)

```
package task3;
import src.Course;
import task1.CourseFormatter;
public class HtmlCourseFormatter implements CourseFormatter {
    @Override
    public String format(Course course) {
        return "<b>" + course.title() + "</b>";
    }
}
```

Листинг 20 — task4/EmailNotificationSender.java (задание 4)

```
package task4;
public class EmailNotificationSender extends AbstractNotificationSender {
    @Override
    protected void doSend(String recipient, String message) {
        System.out.println("email to " + recipient + ": " + message);
    }
}
```

Листинг 21 — task4/ConsoleNotificationSender.java (задание 4)

```
package task4;
public class ConsoleNotificationSender extends AbstractNotificationSender
{
    @Override
    protected void doSend(String recipient, String message) {
        System.out.println("console[" + recipient + "]: " + message);
    }
}
```

Листинг 22 — task8/CourseReadRepository.java (задание 8)

```
package task8;
import src.Course;
import java.util.Optional;
public interface CourseReadRepository {
    Optional<Course> findById(long id);
}
```

Листинг 23 — task8/CourseWriteRepository.java (задание 8)

```
package task8;
import src.Course;
public interface CourseWriteRepository {
    void save(Course course);
}
```

Листинг 24 — task8/ReadOnlyCourseRepository.java (задание 8)

```
package task8;
import src.Course;
import java.util.Optional;
public class ReadOnlyCourseRepository implements CourseReadRepository {
    private final CourseReadRepository delegate;
    public ReadOnlyCourseRepository(CourseReadRepository delegate) {
        this.delegate = delegate;
    }
    @Override
    public Optional<Course> findById(long id) {
        return delegate.findById(id);
    }
}
```

Листинг 25 — task15/AllOf.java (задание 15)

```
package task15;
import src.Course;
import src.Student;
import task12.Accepted;
```

```
import task12.EnrollmentResult;
public class AllOf implements Rule {
    private final Rule[] rules;
    public AllOf(Rule... rules) { this.rules = rules; }
    @Override
    public EnrollmentResult check(Student student, Course course) {
        EnrollmentResult lastAccepted = new Accepted(student.id(),
course.id());
        for (Rule rule : rules) {
            EnrollmentResult result = rule.check(student, course);
            if (result instanceof Accepted accepted) {
                lastAccepted = accepted;
            } else {
                return result;
            }
        }
        return lastAccepted;
    }
}
```

Листинг 26 — task15/AnyOf.java (задание 15)

```
package task15;
import src.Course;
import src.Student;
import task12.Accepted;
import task12.EnrollmentResult;
import task12.Rejected;
public class AnyOf implements Rule {
    private final Rule[] rules;
    public AnyOf(Rule... rules) { this.rules = rules; }
    @Override
    public EnrollmentResult check(Student student, Course course) {
        EnrollmentResult firstRejected = null;
        for (Rule rule : rules) {
            EnrollmentResult result = rule.check(student, course);
            if (result instanceof Accepted) {
                return result;
            }
            if (firstRejected == null) {
                firstRejected = result;
            }
        }
        return firstRejected != null ? firstRejected : new
Rejected(student.id(), course.id(), "no rules configured");
    }
}
```